

Experiment No: 3

Implement a program using Diffie Hellman key exchange algorithm

Name: Rutuja Dundesh Mane

Roll No: A44

Div: A

PRN: 2324000718

ServerDHA.java:

```
import java.awt.*;
import java.io.*;
import java.net.*;
import java.security.*;
import java.security.spec.X509EncodedKeySpec;
import java.util.*;
import javax.crypto.*;
import javax.crypto.spec.SecretKeySpec;
import javax.swing.*;

public class ServerDHA extends JFrame {

    private ServerSocket serverSocket;
    private JTextArea logArea;
    private Map<String, String> qpDatabase;

    public ServerDHA() {
        setTitle("Exam Server - Diffie Hellman");
        setSize(600, 400);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        logArea = new JTextArea();
        logArea.setEditable(false);
        add(new JScrollPane(logArea), BorderLayout.CENTER);

        JButton startBtn = new JButton("Start Server (8080)");
        startBtn.addActionListener(e -> startServer());
        add(startBtn, BorderLayout.SOUTH);

        qpDatabase = new HashMap<>();
        qpDatabase.put("10-CS101-1",
            "Q1. What is Java?\nQ2. Explain OOP concepts.\nQ3. Write features of Java.");

        setVisible(true);
    }

    private void startServer() {
```

```

new Thread(() -> {
    try {
        serverSocket = new ServerSocket(8080);
        log("Server started on port 8080...");

        while (true) {
            Socket client = serverSocket.accept();
            log("Client connected");
            handleClient(client);
        }
    } catch (Exception e) {
        log("Server Error: " + e.getMessage());
    }
}).start();
}

private void handleClient(Socket client) {
    new Thread(() -> {
        try {
            BufferedReader in = new BufferedReader(
                new InputStreamReader(client.getInputStream()));
            PrintWriter out = new PrintWriter(
                client.getOutputStream(), true);

            String cls = in.readLine();
            String courseCode = in.readLine();
            in.readLine();
            String sem = in.readLine();

            String keyDB = cls + "-" + courseCode + "-" + sem;
            String qp = qpDatabase.get(keyDB);

            if (qp == null) {
                out.println("ERROR");
                out.println("Question Paper not found");
                return;
            }

            KeyPairGenerator keyPairGen =
                KeyPairGenerator.getInstance("DiffieHellman");
            keyPairGen.initialize(2048);
            KeyPair serverKP = keyPairGen.generateKeyPair();

            byte[] serverPub = serverKP.getPublic().getEncoded();
            out.println(Base64.getEncoder().encodeToString(serverPub));

```

```

byte[] clientPubBytes =
    Base64.getDecoder().decode(in.readLine());
KeyFactory keyFactory =
    KeyFactory.getInstance("DiffieHellman");
PublicKey clientPubKey =
    keyFactory.generatePublic(
        new X509EncodedKeySpec(clientPubBytes));

KeyAgreement keyAgree =
    KeyAgreement.getInstance("DiffieHellman");
keyAgree.init(serverKP.getPrivate());
keyAgree.doPhase(clientPubKey, true);

byte[] sharedSecret = keyAgree.generateSecret();

SecretKey aesKey =
    new SecretKeySpec(sharedSecret, 0, 16, "AES");

Cipher cipher = Cipher.getInstance("AES");
cipher.init(Cipher.ENCRYPT_MODE, aesKey);

byte[] encryptedQP = cipher.doFinal(qp.getBytes());

out.println("OK");
out.println(Base64.getEncoder().encodeToString(encryptedQP));

log("QP encrypted using Diffie-Hellman + AES");

} catch (Exception e) {
    log("Client Error: " + e.getMessage());
}
}).start();
}

private void log(String msg) {
    logArea.append(msg + "\n");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(ServerDHA::new);
}
}

```

ClientDHA.java:

```

import java.awt.*;
import java.io.*;
import java.net.*;

```

```

import java.nio.file.*;
import java.security.*;
import java.security.spec.*;
import java.util.Base64;
import javax.crypto.*;
import javax.crypto.spec.SecretKeySpec;
import javax.swing.*;

public class ClientDHA extends JFrame {

    private JTextField classField, courseCodeField, semField;
    private JTextArea qpArea;
    private String encryptedQP;
    private SecretKey aesKey;

    public ClientDHA() {
        setTitle("Exam Client - Diffie Hellman");
        setSize(500, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        JPanel inputPanel = new JPanel(new GridLayout(4, 2));

        inputPanel.add(new JLabel("Class:"));
        classField = new JTextField("10");
        inputPanel.add(classField);

        inputPanel.add(new JLabel("Course Code:"));
        courseCodeField = new JTextField("CS101");
        inputPanel.add(courseCodeField);

        inputPanel.add(new JLabel("Semester:"));
        semField = new JTextField("1");
        inputPanel.add(semField);

        JButton requestBtn = new JButton("Request QP");
        requestBtn.addActionListener(e -> requestQP());
        inputPanel.add(requestBtn);

        JButton decryptBtn = new JButton("Decrypt & Download");
        decryptBtn.addActionListener(e -> decryptQP());
        inputPanel.add(decryptBtn);

        add(inputPanel, BorderLayout.NORTH);

        qpArea = new JTextArea();
        qpArea.setEditable(false);
        add(new JScrollPane(qpArea), BorderLayout.CENTER);

        setVisible(true);
    }
}

```

```

private void requestQP() {
    try {

        Socket socket = new Socket("localhost", 8080);
        BufferedReader in =
            new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out =
            new PrintWriter(socket.getOutputStream(), true);

        out.println(classField.getText());
        out.println(courseCodeField.getText());
        out.println("Dummy");
        out.println(semField.getText());

        byte[] serverPubBytes =
            Base64.getDecoder().decode(in.readLine());

        KeyFactory keyFactory =
            KeyFactory.getInstance("DiffieHellman");
        PublicKey serverPubKey =
            keyFactory.generatePublic(
                new X509EncodedKeySpec(serverPubBytes));

        KeyPairGenerator keyPairGen =
            KeyPairGenerator.getInstance("DiffieHellman");
        keyPairGen.initialize(2048);
        KeyPair clientKP = keyPairGen.generateKeyPair();

        out.println(Base64.getEncoder().encodeToString(
            clientKP.getPublic().getEncoded()));

        KeyAgreement keyAgree =
            KeyAgreement.getInstance("DiffieHellman");
        keyAgree.init(clientKP.getPrivate());
        keyAgree.doPhase(serverPubKey, true);

        byte[] sharedSecret = keyAgree.generateSecret();

        aesKey = new SecretKeySpec(sharedSecret, 0, 16, "AES");

        String status = in.readLine();
        if (!"OK".equals(status)) {

```

```

        qpArea.setText("Error receiving QP");
        return;
    }

    encryptedQP = in.readLine();
    qpArea.setText("Encrypted QP:\n\n" + encryptedQP);

} catch (Exception e) {
    qpArea.setText("Error: " + e.getMessage());
}
}

private void decryptQP() {
    try {
        Cipher cipher = Cipher.getInstance("AES");
        cipher.init(Cipher.DECRYPT_MODE, aesKey);

        byte[] decryptedBytes =
            cipher.doFinal(Base64.getDecoder().decode(encryptedQP));

        String decrypted = new String(decryptedBytes);
        qpArea.setText("DECRYPTED QP:\n\n" + decrypted);

        Files.write(Paths.get("QuestionPaper.txt"), decrypted.getBytes());
        JOptionPane.showMessageDialog(this, "QP Downloaded");

    } catch (Exception e) {
        qpArea.setText("Decryption Error: " + e.getMessage());
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(ClientDHA::new);
}
}

```

Output:

Exam Client - Diffie Hellman

Class:

10

Course Code:

CS101

Semester:

1

Request QP

Decrypt & Download

DECRYPTED QP:
Q1. What is java?
Q2. Explain OOP concepts.
Q3. Write features of java.

Exam Server - Diffie Hellman

Server started on port 8080...
Client connected
QP encrypted using Diffie-Hellman + AES

Start Server (8080)